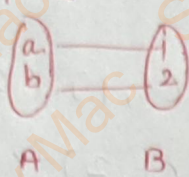
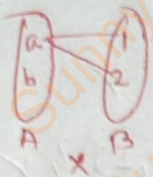
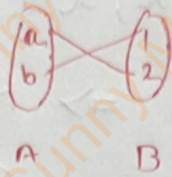


Function 1 A Relation 'f' from a set 'A' to a set 'B' is called function if each element $a \in A$, we can assign unique element of 'B'.

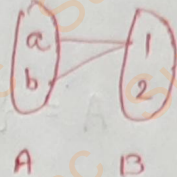
$$A = \{a, b\} \quad B = \{1, 2\}$$



$$\text{Range} = \{1, 2\}$$



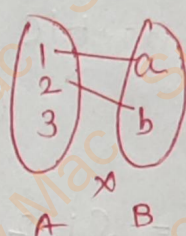
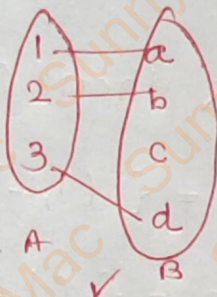
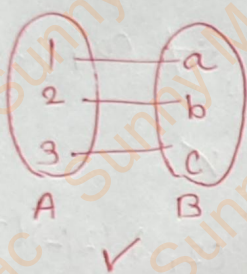
$$\text{Range} = \{1, 2\}$$



$$\text{Range} = \{1\}$$

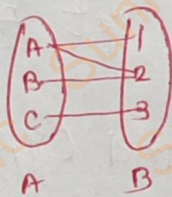
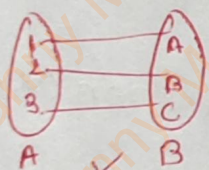
Injective or one-to-one function :

A function 'f' from set 'A' to set 'B' is O-T-O if no two element in A are mapped to same element in B.

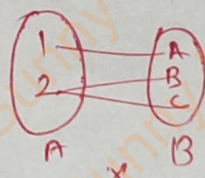
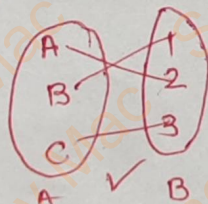
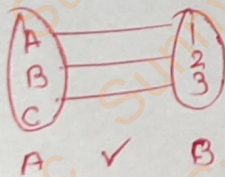


Surjective or onto function :

A function 'f' from set 'A' to set 'B' is onto if each element of 'B' is mapped to atleast one element of 'A'.



Bijection function : if function is one to one and onto.



Reflexive Relation:

— A Relation 'R' on a set 'A' is said to be reflexive if

$$(x, x) \in R \quad \forall x \in A$$

$$A = \{1, 2\} \Rightarrow R_1 = \{(1, 1), (2, 2)\} \quad \checkmark$$

$$R_2 = \{(1, 2), (1, 1), (2, 2)\} \quad \checkmark$$

$$R_3 = \{(1, 2), (2, 2)\} \quad \times \quad (1, 1) \text{ missing}$$

Symmetric Relation:

— A Relation R on a set 'A' is said to be symmetric if

$$(x, y) \in R \text{ then } (y, x) \in R, \quad \forall (x, y) \in A$$

$$A = \{1, 2\} \Rightarrow R_1 = \{(1, 2), (2, 1)\} \quad \checkmark$$

$$A = \{1, 2, 3, 4\} \Rightarrow R_2 = \{(1, 2), (1, 3), (2, 1), (2, 2), (3, 1)\} \quad \checkmark$$

$$R_3 = \{\} \quad \checkmark$$

Transitive Relation:

— A Relation R on a set A is said to be transitive if

$$(x, y) \in R \text{ and } (y, z) \in R \text{ then } (x, z) \in R$$

$$A = \{1, 2, 3\} \Rightarrow R_1 = \{(1, 2), (2, 3), (1, 3)\} \quad \checkmark$$

$$T(n) = aT\left(\frac{n}{b}\right) + O(n^k \log^p n), \quad \left. \begin{array}{l} a \geq 1 \\ b \geq 1 \\ k \geq 0 \end{array} \right\} \begin{array}{l} \text{all} \\ \text{Constant} \end{array}$$

p is a real number.

$\rightarrow$ • $a > b^k$ then $T(n) = O(n^{\log_b a})$
 • $a = b^k$ then check p ,
 $p > -1$ then $T(n) = O(n^{\log_b a} \log^{p+1} n)$
 $p = -1$ then $T(n) = O(n^{\log_b a} \log \log n)$
 $p < -1$ then $T(n) = O(n^{\log_b a})$
 • $a < b^k$ then check p ,
 $p \geq 0$ then $T(n) = O(n^k \log^p n)$
 $p < 0$ then $T(n) = O(n^k)$

⑤ $T(n) = 4T(n/2) + n$
 $a = 4, b = 2, k = 1, p = 0$
 Compare a & b^k
 $a \quad b^k$
 $4 \quad 2^1 \Rightarrow a > b^k$, so $\log_b a$
 $T(n) = O(n^{\log_b a})$
 $= O(n^{\log_2 4})$
 $= O(n^2)$

⑥ $T(n) = 3T(n/2) + n^2$
 $a = 3, b = 2, k = 2, p = 0$
 Compare a & b^k
 $a \quad b^k$
 $3 \quad 2^2$
 $3 < 4 \Rightarrow a < b^k$, so $\Rightarrow$
 Check p .
 $p \geq 0$,
 $T(n) = O(n^k \log^p n)$
 $= O(n^2 \cdot \log^0 n)$
 $= O(n^2)$

$$\textcircled{3} T(n) = 4T\left(\frac{n}{2}\right) + n^2$$

$$a = 4, b = 2, k = 2, p = 0$$

Compare a & b^k

$$a \quad b^k$$

$$4 \quad 2^2$$

$$4 = 4 \Rightarrow a = b^k, \text{ check } p$$

$$p > -1$$

$$\therefore T(n) = \Theta(n^{\log_b a \cdot \log^{p+1} n})$$

$$= \Theta(n^{\log_2 4 \cdot \log^{0+1} n})$$

$$= \Theta(n^2 \cdot \log n)$$

$$\textcircled{4} T(n) = T\left(\frac{n}{2}\right) + n^2$$

$$a = 1, b = 2, k = 2, p = 0$$

Compare,

$$a \quad b^k$$

$$1 \quad 2^2$$

$$1 < 4 \Rightarrow a < b^k, \text{ check } p$$

$$p > = 0$$

$$\therefore T(n) = \Theta(n^k \log^p n)$$

$$= \Theta(n^2 \cdot \log^0 n)$$

$$= \Theta(n^2)$$

$$\textcircled{5} T(n) = 2^n T\left(\frac{n}{2}\right) + n^n$$

a is not constant. Can not solve with master method.

$$\textcircled{6} T(n) = 0.5T\left(\frac{n}{2}\right) + \frac{1}{n}$$

$a < 0$ and a must be ≥ 1 . Can not solve using master method.

$$\textcircled{7} T(n) = 64T\left(\frac{n}{8}\right) - n^2 \log n$$

- given formula do not match with master method std formula

- so we can not solve using master method

Multiplying Large Integers

$$0981 \times 1234$$

$$\text{Step 1: } a_1 = 09 \quad b_1 = 12$$

$$a_0 = 81 \quad b_0 = 34$$

$$\text{Step 2: } c_2 = a_1 \times b_1$$

$$= 09 \times 12$$

$$c_2 = 108$$

$$c_0 = a_0 \times b_0$$

$$= 81 \times 34$$

$$c_0 = 2754$$

$$\text{Step 3: } c_1 = (a_1 + a_0) * (b_1 + b_0) - (c_2 + c_0)$$

$$= (9 + 81) * (12 + 34) - (108 + 2754)$$

$$= 4140 - 2862$$

$$c_1 = 1278$$

$$\text{Step 4: } c = c_2 10^n + c_1 10^{n/2} + c_0, \quad n \text{ is total no. of digits}$$

$$= c_2 10^4 + c_1 10^2 + c_0$$

$$= 108 \times 10000 + 1278 \times 100 + 2754$$

$$= 1210554$$

$$\text{Time Complexity} = O(n^{1.58})$$

Matrix multiplication

$$C = A \times B, \quad \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \times \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$$

$$S_1 = (A_{11} + A_{22}) (B_{11} + B_{22})$$

$$S_2 = (A_{21} + A_{22}) \times B_{11}$$

$$S_3 = A_{11} \times (B_{12} - B_{22})$$

$$S_4 = A_{22} \times (B_{21} - B_{11})$$

$$S_5 = (A_{11} + A_{12}) \times B_{22}$$

$$S_6 = (A_{21} - A_{11}) \times (B_{11} + B_{12})$$

$$S_7 = (A_{12} - A_{22}) \times (B_{21} + B_{22})$$

$$C_{11} = S_1 + S_4 - S_5 + S_7$$

$$C_{12} = S_3 + S_5$$

$$C_{21} = S_2 + S_4$$

$$C_{22} = S_1 + S_3 - S_2 + S_6$$

$$\text{Time complexity} = O(n^{2.81})$$

Knapsack problem (Fractional)

$m = 15$

p	10	5	15	7	6	18	3
w	2	3	5	7	1	4	1

Step 1: Find P/W ratio.

Step 2: Set them in descending order of P/W ratio.

Object	01	02	03	04	05	06	07
p	10	5	15	7	6	18	3
w	2	3	5	7	1	4	1
P/w	5	1.67	3	1	6	4.5	3
	(2)	(6)	(4)	(7)	(1)	(3)	(5)

Re-arrange $\Rightarrow$

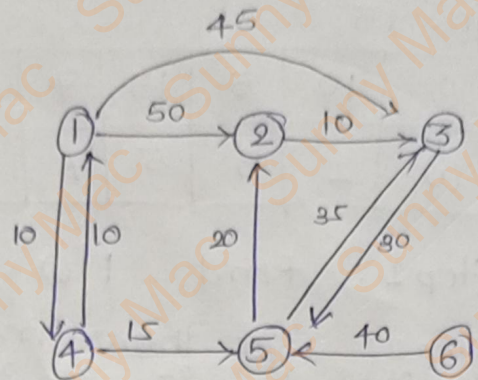
p	6	10	18	15	3	5	7
w	1	2	4	5	1	3	7

Object No	P	w	Remaining w
-	-	-	15
5	6	1	$15 - 01 = 14$
1	10	2	$14 - 02 = 12$
6	18	4	$12 - 04 = 08$
3	15	5	$08 - 05 = 03$
7	3	1	$03 - 01 = 02$
2	3.33	2	$02 - 02 = 00$
	<u>37.33</u>		

$$\begin{array}{c} w \\ 3 \\ 2 \end{array} \rightarrow \begin{array}{c} p \\ 5 \\ ? \end{array} \Rightarrow \frac{10}{3} = 3.33$$

Graph - one pair shortest path - Dijkstra's Algo

	2	3	4	5	6
1	50	45	(10)	∞	∞
4	50	45	10	(25)	∞
5	(45)	45	10	25	∞
2	45	(45)	10	25	∞
3	45	45	10	25	(∞)
6	45	45	10	25	∞



1-2 $\Rightarrow$ 45 (1-4-5-2)

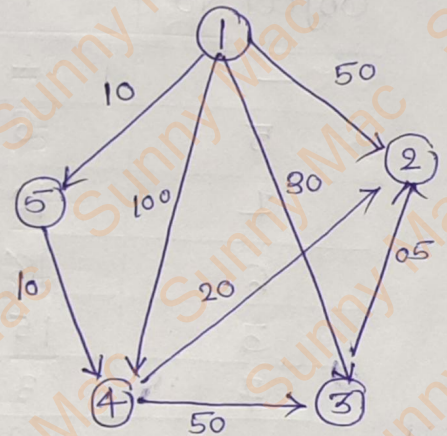
1-3 $\Rightarrow$ 45 (1-3)

1-4 $\Rightarrow$ 10 (1-4)

1-5 $\Rightarrow$ 25 (1-4-5)

1-6 $\Rightarrow$ No path -

	2	3	4	5
1	50	30	100	(10)
5	50	30	(20)	10
4	40	(30)	20	10
3	(35)	30	20	10
2	35	30	20	10



Dynamic method	Greedy method
- choice is made at each steps. - Less efficient with compare to Greedy - It gives optimal solution - Example: 0/1 knapsack	- choice is made once only and all subproblems will be solved using same method - more efficient compared to dynamic - It gives feasible solution - Example: Fractional knapsack

Binomial Coefficient

$$\leadsto (a+b)^2 = 1a^2 + 2ab + 1b^2$$

$$(a+b)^3 = 1a^3 + 3a^2b + 3ab^2 + 1b^3$$

- Numbers that appears as coefficient of the term in a binomial expression is binomial co.

- Binomial coefficient of n and k is written as

$$(n, k) \text{ or } {}^nC_k \text{ or } \binom{n}{k}$$

$${}^nC_k = \frac{n!}{k!(n-k)!} \Rightarrow {}^5C_3 = \frac{5!}{3!(5-3)!} = 10$$

We can solve this using dynamic

* Find Binomial coefficient of 5C_3

$$n = 5, c = 3$$

$$C[i, j] = C[i-1, j] + C[i-1, j-1]$$

$n \downarrow r \rightarrow$	0	1	2	3
0	1	X		
1	1	1	X	
2	1	2	1	X
3	1	3	3	1
4	1	4	6	4
5	1	5	10	10

(method)

- Fill $(0,0) \Rightarrow 1$ &

Same number $\Rightarrow 1$

- Add diagonal + top element

$$\therefore {}^5C_3 = 10$$

* making change problem.

Total = 11 coin = 1, 5, 6, 8

	0	1	2	3	4	5	6	7	8	9	10	11
1	0	1	2	3	4	5	6	7	8	9	10	11
5	0	1	2	3	4	1	2	3	4	5	2	3
6	0	1	2	3	4	1	1	2	3	4	2	2
8	0	1	2	3	4	1	1	2	1	2	2	2

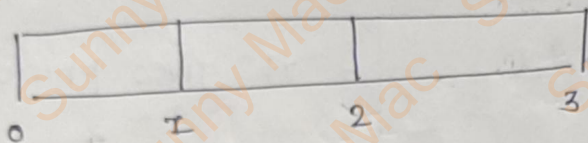
Selected coin 5, 6

Job Scheduling problem

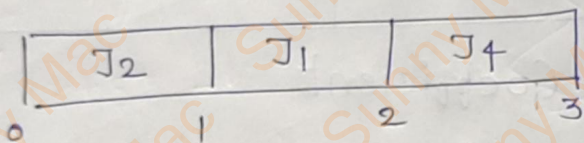
	J ₁	J ₂	J ₃	J ₄	J ₅
profit	20	15	10	5	1
deadline	2	2	1	3	3

Step 1: Set in descending order of profit.

Step 2: Find last possible deadline and make slots.



Step 3: Assign to maximum late possible and if not available, assign to previous possible slot.



Job	Schedule Time	profit
J ₁	1 - 2	20
J ₂	0 - 1	15
J ₃	—	—
J ₄	2 - 3	05
J ₅	—	—

Activity Selection Problem

Activity	a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8
Start	1	0	1	4	2	5	3	4
Stop	3	4	2	6	9	8	5	5

Step 1: Arrange in ascending order of end time.

Activity	a_3	a_1	a_2	a_7	a_8	a_4	a_6	a_5
Start	1	1	0	3	4	4	5	2
Stop	2	3	4	5	5	6	8	9

Diagram showing selection process: a_3 (start 1, stop 2) is selected. a_1 (start 1, stop 3) is rejected because its start is not $\leq$ the end of the last selected activity. a_2 (start 0, stop 4) is rejected because its start is not $\leq$ the end of the last selected activity. a_7 (start 3, stop 5) is selected. a_8 (start 4, stop 5) is rejected because its start is not $\leq$ the end of the last selected activity. a_4 (start 4, stop 6) is rejected because its start is not $\leq$ the end of the last selected activity. a_6 (start 5, stop 8) is selected. a_5 (start 2, stop 9) is rejected because its start is not $\leq$ the end of the last selected activity. The final selected activities are a_3, a_7, a_6 .

Step 2: Select 1st activity & check next start $\leq$ 1st end time

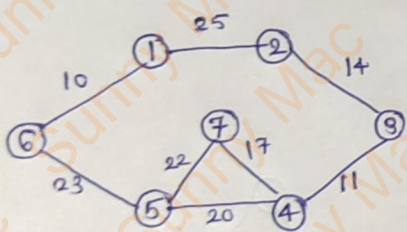
Total 3 activity

$a_3 \Rightarrow 1-2$

$a_7 \Rightarrow 3-5$

$a_6 \Rightarrow 5-8$

Minimum Spanning Tree



A spanning Tree of a graph G is subgraph which is basically a tree and it contains all the vertices of G containing No circuits.

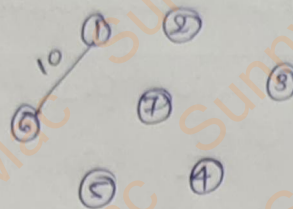
→ A minimum spanning tree of a weighted connected graph G is a spanning tree with minimal or smallest weight.

• Prim's algorithm :

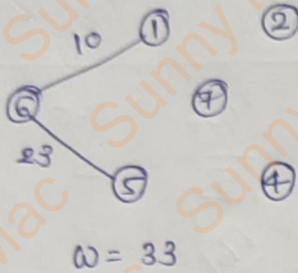
Step 1 :



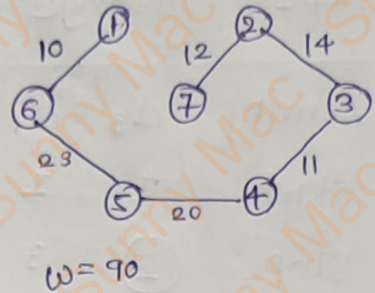
Step 2 :



Step 3 :

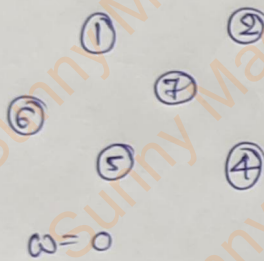


Step n :

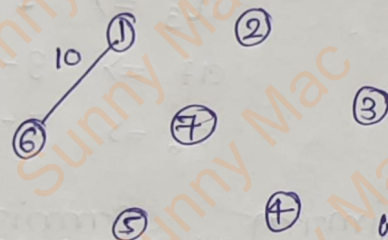


• Kruskal's algorithm :

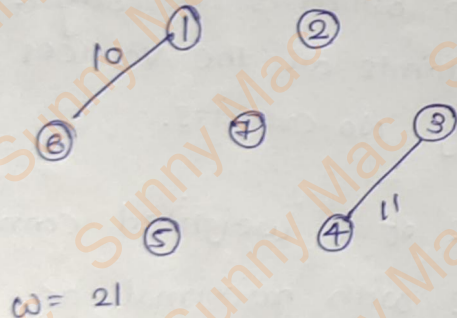
Step 1 :



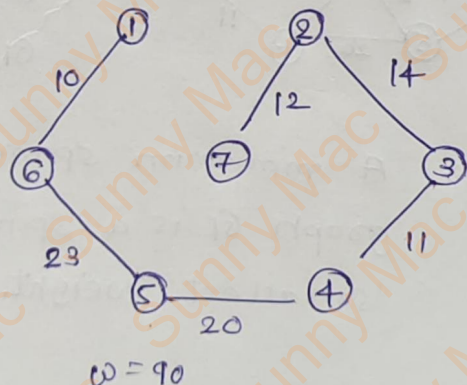
Step 2 :

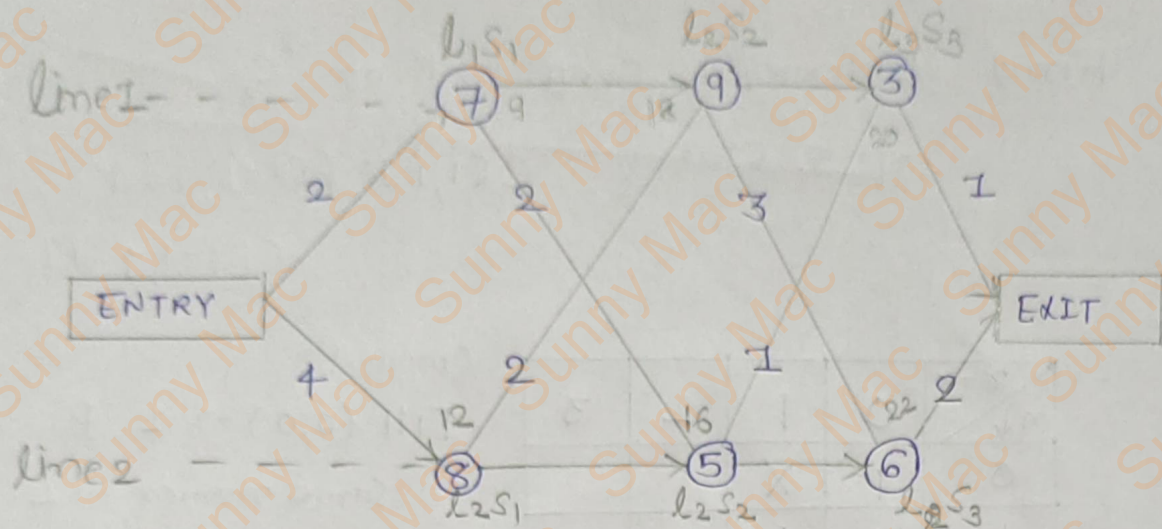


Step 3 :



Step n :





- calculate load on each station & choose the minimum load

$$L_1S_1 = 2 + 7 = 9 \quad L_2S_1 = 4 + 8 = 12$$

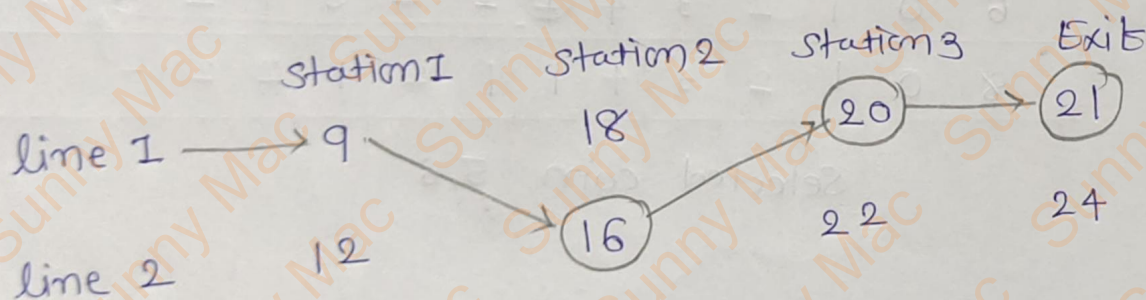
$$L_2S_2 \begin{cases} L_1 \rightarrow 9 + 9 = 18 \\ L_2 \rightarrow 12 + 2 + 9 = 23 \end{cases}$$

$$L_2S_2 \begin{cases} L_1 \rightarrow 9 + 2 + 5 = 16 \\ L_2 \rightarrow 12 + 5 = 17 \end{cases}$$

$$L_1S_3 \begin{cases} L_1 \rightarrow 18 + 3 = 21 \\ L_2 \rightarrow 16 + 1 + 3 = 20 \end{cases}$$

$$L_2S_3 \begin{cases} L_1 \rightarrow 18 + 3 + 6 = 27 \\ L_2 \rightarrow 16 + 6 = 22 \end{cases}$$

$$\text{Exit load} = L_1 \Rightarrow 20 + 1 = 21, \quad L_2 = 22 + 2 = 24$$



* Longest Common Subsequence

abcedaf, acbef

Step 1: Create a table with string 1 & string 2

		a	b	c	d	a	f
a	0	0	1	1	1	1	1
c	0	1	1	2	2	2	2
b	0	1	2	2	2	2	2
c	0	1	2	3	3	3	3
f	0	1	2	3	3	3	4

if $i = j$

count + 1

else

$\max(\text{top}, \text{left})$

length = 4

seq $\Rightarrow$ a, b, c, f

* 0-1 Knapsack problem

Profit = {1, 2, 5, 6} Bag Capacity = 8

Weight = {2, 3, 4, 5}

Step 1: Re-arrange as per weight (ascending order)

Step 2: create a table Bag capacity as coll. & number of items as row.

P	W	Item	0	1	2	3	4	5	6	7	8	→ Bag Capacity (W)
-	-	0	0	0	0	0	0	0	0	0	0	
1	2	1	0	0	1	1	1	1	1	1	1	Check W but put P
2	3	2	0	0	1	(2)	2	3	3	3	3	✓
5	4	3	0	0	1	2	5	5	6	7	7	
6	5	4	0	0	1	(2)	5	6	6	7	(8)	✓

Total W

- 2 product selected
- max profit = 8
- {0, 1, 0, 1} ← product selection in 0, 1 form.

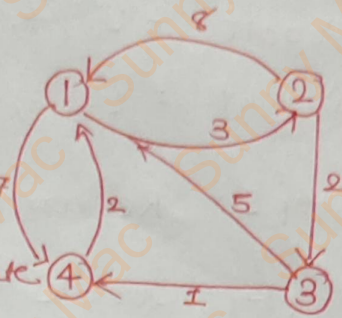
* All pair shortest path.

Start with A_0 .

- A_0 means only direct connection.

- Next A_1 means 1 as intermediate node.

- Next A_2 means 1, 2 as intermediate node.



$$A_0 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 8 & \infty & 7 \\ 8 & 0 & 2 & \infty \\ 5 & \infty & 0 & 1 \\ 2 & \infty & \infty & 0 \end{bmatrix} \end{matrix}$$

$$A_1 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & \infty & 7 \\ 8 & 0 & 2 & 15 \\ 5 & 8 & 0 & 1 \\ 2 & 5 & \infty & 0 \end{bmatrix} \end{matrix}$$

$$A_2 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & 5 & 7 \\ 8 & 0 & 2 & 15 \\ 5 & 8 & 0 & 1 \\ 2 & 5 & 7 & 0 \end{bmatrix} \end{matrix}$$

$$A_3 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & 5 & 7 \\ 7 & 0 & 2 & 3 \\ 5 & 8 & 0 & 1 \\ 2 & 5 & 7 & 0 \end{bmatrix} \end{matrix}$$

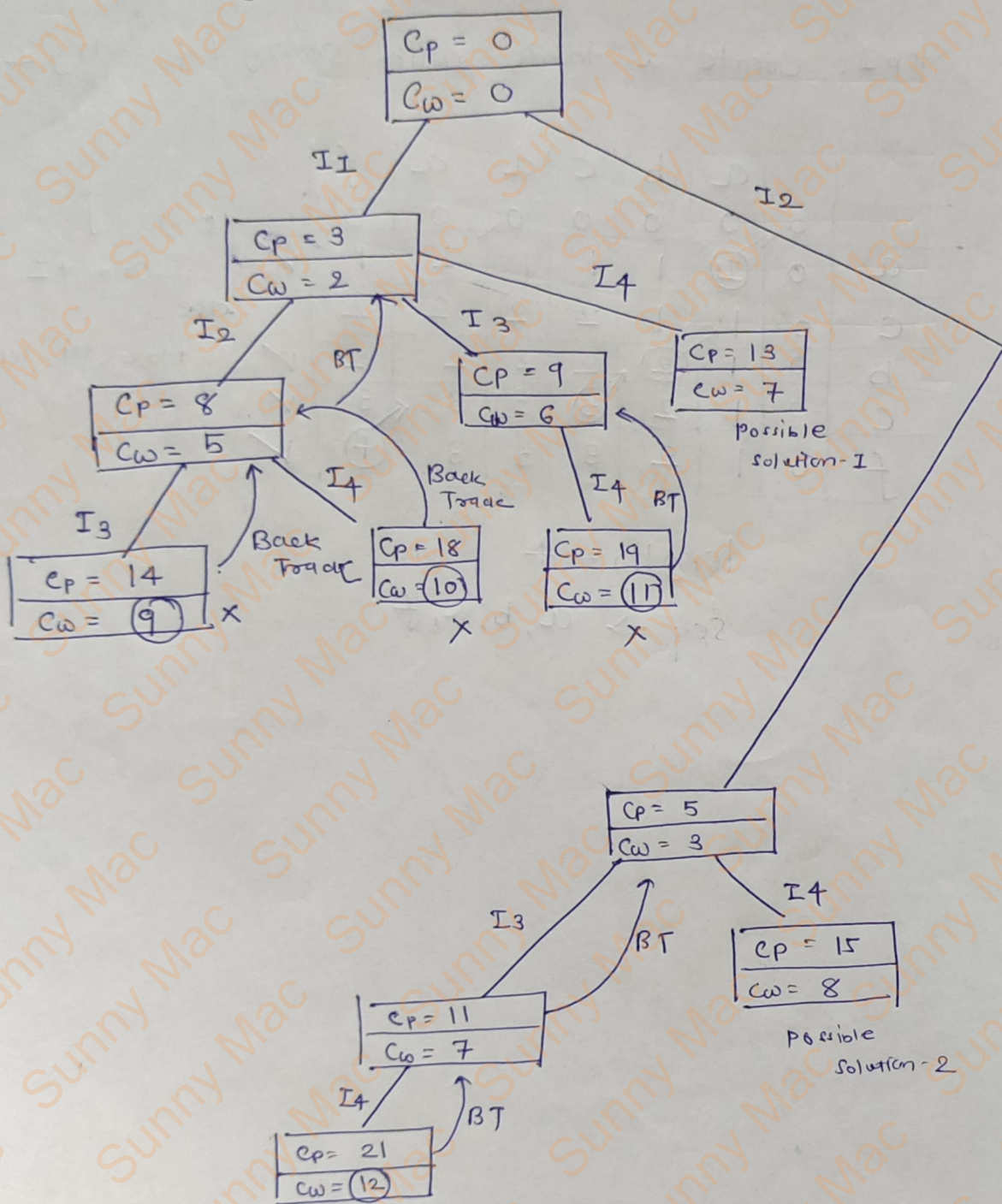
$$A_4 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & 5 & 6 \\ 5 & 0 & 2 & 3 \\ 3 & 6 & 0 & 1 \\ 2 & 5 & 7 & 0 \end{bmatrix} \end{matrix}$$

Complexity $O(n^3)$

* Profit $\{3, 5, 6, 10\}$, weight $= \{2, 3, 4, 5\}$

Solve using Branch & Bound method

Bag Capacity = 8



Amortized Analysis

→ Amortized analysis is a worst case analysis of a sequence of operations - to obtain a tighter bound on the overall or average cost per operation in the sequence than is obtained by separately analyzing each operation in the sequence.

→ mainly 3 methods are used

- Aggregate analysis
- Accounting method / Banker's algorithm
- Potential method / physicist's method

Aggregate

Example: Let's consider an extensible array that can store numbers using operation Add. If the array is full or no cell left, it will double the size and copy all old data to new array.

Insert 11

11

 size = 1

Insert 12

11	12
----	----

 size = 2

Insert 13

11	12	13	
----	----	----	--

 size = 4

Insert 14

11	12	13	14
----	----	----	----

 size = 4

Insert 15

11	12	13	14	15			
----	----	----	----	----	--	--	--

 size = 8

Let's consider i as insertion number, S_i as size of table & C_i for cost.

$$C_i = \begin{cases} i & \text{if } i-1 \text{ is power of } 2 \\ 1 & \text{otherwise} \end{cases}$$

i	1	2	3	4	5	6	7	8	9	10
Si	1	2	4	4	8	8	8	8	16	16
Ci	1	2	3	1	5	1	1	1	9	1

Accounting (Banker's) method

- Find Highest possible cost
 - change some more cost and save it for future use.
- ~> Consider i as insertion operation, Si as size of table, Ci as actual cost, C' as new cost and bi as balance.

i	1	2	3	4	5	6	7	8	9	10
Si	1	2	4	4	8	8	8	8	16	16
Ci	1	2	3	1	5	1	1	1	9	1
C'	3	3	3	3	3	3	3	3	3	3
bi	2	3	3	5	3	5	7	9	3	4

Potential (Physicist's) method

- Suppose we define a potential function ϕ ("phi"),
 $\phi(h_0) = 0$, where h_0 is the initial state
 $\phi(h_t) \geq 0$ for all states
then we can define the amortized time
 $c + \phi(h') - \phi(h)$.